ДЗ 5. RL для вызова функций

В работе сравниваются три варианта RL для Qwen/Qwen3.5-2B: GRPO с точной наградой, GRPO с составной наградой и GRPO Done Right с той же составной наградой. Все запуски начинаются с исходной модели. Для обучения используется только ToolACE, без SFT.

Данные

В Team-ACE/ToolACE 11 300 диалогов. Сначала 565 целых диалогов отложены в validation, а уже потом каждый диалог разбит на префиксы до ответа ассистента. Так части одного разговора не попадают одновременно в train и validation. Взяты ответы после сообщений пользователя; предыдущие сообщения и результаты инструментов остаются в истории. Эталон нужен только для награды, в запрос модели он не добавляется.

После фильтрации осталось 11154 примера для обучения и 602 для validation. В train 9008 примеров с вызовом и 2146 без вызова. Медианная длина промпта до фильтрации - 701 токен, 95-й процентиль - около 1404. Примеры длиннее 2048 токенов, пустые ответы и повторы промптов убраны.

Описания функций приведены к JSON Schema. Недопустимые символы в именах заменены подчёркиваниями, эти же замены применены к эталонам. Диалоги с неподдерживаемой схемой или неразбираемыми вызовами исключены. Аргументы оставленных эталонов проверены по схемам. Точных совпадений вопросов с BFCL после нормализации не найдено. Версии данных, состав сплитов и причины исключения сохранены в results.

Для быстрой проверки на ToolACE взяты 192 отложенных префикса: 64 без вызова, 64 с одним вызовом и 64 с несколькими. Они одинаковы для всех моделей.

Обучение и награда

Точная награда равна 1, если совпал весь набор вызовов и аргументов, иначе 0. Порядок вызовов не важен, повторы учитываются. Числа 1 и 1.0 считаются равными, у строк убираются крайние пробелы. Для составной награды добавлены частичные оценки:

$R_{exact} = E$
$R_{shaped} = 0.1V + 0.2F_{name} + 0.2F_{args} + 0.5E$

Здесь E - точное совпадение, V - корректная разметка и соответствие схемам. F_{name} - F1 по именам функций, F_{args} - F1 по тройкам «функция, аргумент, значение», в обоих случаях с учётом повторов. Если аргументов нет с обеих сторон, $F_{args} = F_{name}$. Пустой, обрезанный или неправильно размеченный ответ получает 0. Если вызов нужен, а его нет, награда тоже 0. Если вызов не нужен, обе награды дают 1 за непустой ответ без вызова. Полезность самого текста при этом не оценивается. Обрезанные генерации также исключаются из policy loss.

Например, при эталоне weather(city='Paris') такой же вызов получает 1 по обеим наградам. За weather(city='London') точная награда даёт 0, а составная - 0.3: функция и схема верны, но город другой. Незакрытый tool_call даёт 0. Это примеры проверки награды, в обучение они не входят. Для разбора используется тот же Qwen3CoderToolParser, что и в evaluate.py.

В первом запуске проверяется, хватит ли награды за полное совпадение. Во втором меняется только награда: частичные оценки должны давать сигнал и тогда, когда ни один из четырёх ответов не полностью верен. В третьем проверяется

GRPO Done Right. Он убирает деление advantage на std группы, а сумму по токенам делит на фиксированную длину 512, умноженную на число ответов. Это две совместные замены нормировки. У обычного GRPO токены сначала усредняются внутри каждого ответа.

Каждый запуск обучается 300 шагов на одной A100 80 GB через TRL 1.16.0. Используется LoRA: r=16, alpha=32, dropout=0, проекции q/k/v/o, gate/up/down, in_proj_qkv/z/a/b и out_proj. Исходные веса BF16, адаптеры FP32; включены SDPA, TF32 и gradient checkpointing. Оптимизатор AdamW fused, lr=2e-5, warmup=10 шагов, затем постоянный lr. Weight decay=0.01, gradient clipping=0.3, beta=0.02, clipping ratio=0.2.

Batch size - 8, gradient accumulation - 4, по 4 генерации на запрос. В сумме это 32 ответа на 8 запросов за шаг; num_iterations=1. При генерации temperature=0.8, top_p=0.95, top_k=0, максимум 512 новых токенов. Seed данных и Trainer - 42, seed rollout engine - 0, как по умолчанию в TRL для одного процесса. Для всех конфигураций заранее выбран последний, 300-й шаг. Короткие пробные запуски использовались только для проверки кода и памяти.

Обучаемых параметров в каждом варианте 16 819 200. Время с инициализацией trainer: GRPO exact - 5618.6 с, GRPO shaped - 3722.5 с, GRPO Done Right - 3583.9 с. Пиковая allocated-память PyTorch - 35.70, 35.50 и 35.73 GiB соответственно, вместе с rollout engine. Это не замер всей памяти через NVML. По этим временам отдельно сравнивать скорость алгоритмов не стоит: в них входят инициализация и влияние кэшей.

Результаты

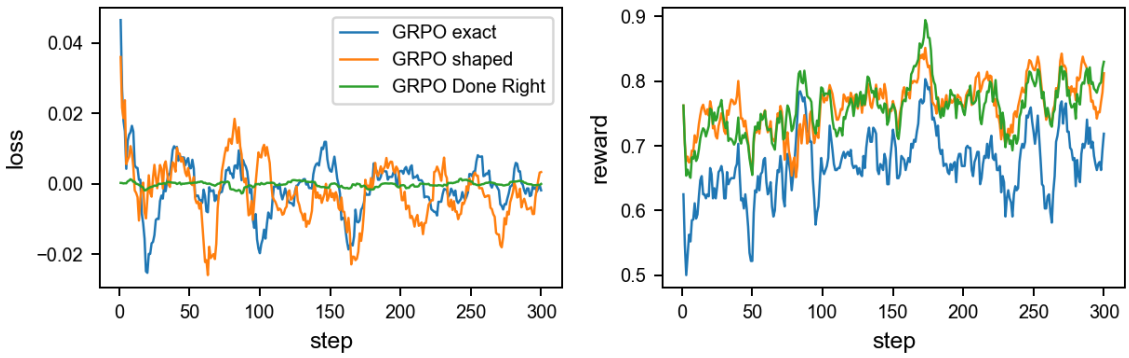
Модели оценены исходным evaluate.py на тех же 350 примерах BFCL. Параметры не менялись: greedy, temperature=0, максимум 1024 новых токена, thinking отключён, seed=42. Токенизатор исходный, LoRA перед оценкой объединена с весами модели. Overall - среднее по пяти группам; в Multiple и Irrelevance сначала усредняются их две категории. Поэтому overall отличается от обычной доли правильных ответов среди всех 350 примеров.

Модель	Overall	Simple	Multiple	Parallel	Parallel + Multiple	Irrelevance
Base	68.93	68.00	78.67	56.00	80.00	62.00
GRPO exact	70.53	62.00	74.67	48.00	84.00	84.00
GRPO shaped	78.93	86.00	80.67	80.00	88.00	60.00
GRPO Done Right	78.40	80.00	82.00	74.00	88.00	68.00

В таблице значения в процентах. Лучший overall у GRPO shaped: 78.93% против 68.93% у base. Прирост - 10 п.п., 95% парный bootstrap-интервал [4.80; 15.47] п.п. Для bootstrap сделано 3000 пересэмплирований внутри семи исходных категорий с сохранением весов overall.

Для этой задачи предпочтителен GRPO с составной наградой, хотя GRPO Done Right почти не отстаёт: разница 0.53 п.п., интервал [-3.07; 4.00] п.п. Уверенного различия между ними здесь нет. При этом GRPO Done Right лучше в Irrelevance: 68% против 60%. У точной награды всего +1.60 п.п. к base, а интервал [-2.93; 6.13] п.п. включает ноль. Base и GRPO exact правильно решили одинаковые 251 из 350 примеров. Разный overall получился из-за разных весов групп.

Ниже loss и reward всех трёх запусков, сглаженные по 10 шагам. У GRPO Done Right другая нормировка loss, поэтому меньшие значения сами по себе не означают лучшее обучение.



Ответы на вопросы

1. У полного совпадения слишком мало промежуточного сигнала: одна ошибка в аргументе обнуляет ответ. Составная награда позволяет поощрить хотя бы правильную функцию и часть аргументов. Это согласуется с логами: доля групп с одинаковыми наградами у exact - 82.75%, у shaped - 75.54%. При этом частичная оценка аргументов не различает их перестановку между несколькими вызовами одной функции; полное совпадение Е это различает.

2. Больше всего помогла замена точной награды на составную: +8.40 п.п. overall при том же GRPO, интервал [3.87; 13.20] п.п. Веса на старте, данные, seed, LoRA, lr и число шагов одинаковые, меняется только награда. У GRPO Done Right одновременно меняются две нормировки, так что их отдельные эффекты по этому запуску разделить нельзя.

3. Рост reward не всегда совпадает с BFCL. Средняя награда за первые и последние 20 шагов: exact - 0.627 и 0.723, shaped - 0.737 и 0.802, GRPO Done Right - 0.717 и 0.826. У GRPO Done Right при той же составной награде итоговый reward выше, но BFCL чуть ниже. При этом разница BFCL мала относительно интервала неопределённости. Награды разных форм напрямую сравнивать нельзя. Промежуточный BFCL не замерялся.

На ToolACE validation точный exact match у GRPO exact - 77.60%, у shaped - 73.44%, у GRPO Done Right - 75.00%, у base - 63.02%. По BFCL порядок exact и shaped обратный. Одна из причин - разный состав оценки: в validation треть примеров без вызова, а у BFCL группа Irrelevance весит одну пятую. Exact лучше отказывается от лишних вызовов (84% против 60% в BFCL), зато shaped лучше в Parallel (80% против 48%).

4. Если все ответы на запрос получили одинаковую награду, R_i - mean(R) = 0 и advantage равен нулю. Policy loss от этой группы не даёт обучающего сигнала. При beta=0.02 при этом может оставаться градиент KL-штрафа к исходной модели.

5. У лучшего варианта GRPO shaped остались ошибки и в аргументах, и в решении о вызове. Пять примеров из predictions.jsonl:

simple_150: для вклада под 3% в calculate_roi передано annual_interest_rate=3 вместо 0.03. Функция правильная, но процент не переведён в долю.

multiple_76: на запрос о бронзовой скульптуре лошади модель перечисляет материалы текстом. Вызов sculpture.create_custom пропущен. Сам запрос можно прочитать и как вопрос о материалах, но эталон BFCL ожидает создание скульптуры.

parallel_181: требовались цены Apple за неделю, Microsoft за две, Amazon за три и Tesla за месяц. Модель сделала четыре get_stock_prices, но в каждом передала сразу все компании. Число вызовов верно, соответствие компаний периодам потеряно.

parallel_multiple_67: в math.roots.polynomial передан массив [6, -3, 2, -1, 1], а checker требует float: [6.0, -3.0, 2.0, -1.0, 1.0]. Числа по смыслу те же, но ответ отклонён. Это пример расхождения reward и BFCL: числовая нормализация в награде такую разницу не учитывает.

irrelevance_233: на вопрос о переводе $20 в фунты вызвана weather_in_location для Лондона. Доступен только инструмент погоды, так что это лишний вызов: подходящей функции для запроса нет.

В целом составная награда лучше подошла для нужных вызовов, а точная сильнее смещает модель к отказу от них. Результат относится к этим 350 примерам и одному seed на вариант. Bootstrap оценивает разброс тестовой выборки, но не разброс повторных обучений. Чтобы надёжнее сравнить GRPO shaped и GRPO Done Right, потребовались бы повторные запуски и отдельный тест.

Материалы: модель, ToolACE, TRL, условие.